



UVM Test Plan CRC BLE

You're viewing user-generated content that may be unverified or unsafe. [Report](#)

Here's a basic **UVM test plan** for a **BLE CRC DUT**.

UVM Test Plan for BLE CRC DUT

1. Objective

Verify that the **BLE CRC DUT** correctly computes the **24-bit BLE CRC** for valid packet data streams, using the proper initialization value and bit processing order, across normal, boundary, and error-related scenarios.

2. DUT Description

The DUT is assumed to:

- Accept BLE packet bits or bytes as input
 - Use a **24-bit CRC polynomial** defined by BLE
 - Support a configurable or fixed **CRC initialization value**
 - Produce a **24-bit CRC result**
 - Optionally indicate completion with a valid/done signal
-

3. Verification Goals

The verification environment should confirm:

- Correct CRC generation for known BLE packet examples
 - Correct handling of different payload lengths
 - Correct handling of edge cases such as zero-length or minimum-length payloads
 - Correct reset behavior
 - Correct operation across randomized packet content
 - Bit ordering and byte ordering match BLE spec expectations
 - No unexpected state retention between transactions
-

4. Assumptions

This plan assumes the DUT interface includes some subset of:

- `rst_n`
- `start`
- `data_in`
- `data_valid`
- `crc_init`
- `done`
- `crc_out`

If the DUT is byte-based instead of bit-based, the same plan still applies with adapted stimulus packing.

5. Testbench Architecture

Basic UVM components:

- **sequence item**
 - packet payload
 - payload length
 - CRC init value
 - expected CRC
 - **sequence**
 - generates directed and randomized BLE packets
 - **driver**
 - drives packet data into DUT
 - **monitor**
 - captures DUT input/output activity
 - **scoreboard**
 - computes reference CRC and compares against DUT output
 - **coverage collector**
 - tracks payload lengths, patterns, reset interactions, and corner cases
 - **agent**
 - active agent for stimulus, passive monitoring optional
 - **env**
 - integrates agent, scoreboard, and coverage
 - **test**
 - configures sequences for each scenario
-

6. Reference Model

A golden reference model shall compute BLE CRC independently from the DUT.

The model should:

- Implement the official BLE CRC polynomial
- Use the correct BLE bit ordering
- Support configurable 24-bit CRC initialization
- Produce expected CRC for scoreboard comparison

This reference model is the key checker in the scoreboard.

7. Features to Verify

7.1 Reset

Verify that:

- CRC state resets correctly
- outputs return to expected idle values
- reset during idle behaves correctly
- reset between transactions clears prior state

7.2 CRC Initialization

Verify that:

- default CRC init is correct
- programmed CRC init is correctly loaded
- different init values affect output as expected

7.3 Payload Processing

Verify that:

- single-byte payloads work
- multi-byte payloads work
- minimum and maximum supported payload sizes work
- all-zero, all-one, alternating, and random payload patterns work

7.4 Transaction Control

Verify that:

- `start` launches a new CRC calculation

- `done` asserts correctly
- output CRC is stable when valid
- back-to-back transactions work correctly

7.5 BLE-Specific Behavior

Verify that:

- BLE polynomial is correctly applied
 - BLE bit ordering is correct
 - packet bytes are consumed in the expected order
 - CRC matches published BLE examples when available
-

8. Directed Test List

Test 1: Reset Smoke Test

Purpose:

- Confirm DUT resets correctly

Checks:

- CRC register cleared or set to expected reset state
- no false done/output activity

Test 2: Single Known Packet

Purpose:

- Compare DUT against one hand-checked BLE example

Checks:

- exact CRC match

Test 3: Multiple Known Packets

Purpose:

- Verify against several directed payloads

Patterns:

- all zeros
- all ones
- alternating `10101010`
- alternating `01010101`
- incrementing bytes

Test 4: Zero-Length Payload

Purpose:

- Verify CRC behavior for empty input, if supported

Checks:

- result equals expected CRC from init-only state

Test 5: Minimum Payload

Purpose:

- Verify smallest legal BLE payload size

Test 6: Maximum Payload

Purpose:

- Verify largest supported payload size

Test 7: Different CRC Init Values

Purpose:

- Confirm DUT correctly uses multiple init seeds

Test 8: Back-to-Back Transactions

Purpose:

- Ensure no state leakage between packets

Test 9: Mid-Transaction Reset

Purpose:

- Verify reset recovery during active CRC computation

Test 10: Random Payload Regression

Purpose:

- Stress DUT with many randomized packets and lengths

9. Randomization Strategy

Randomize:

- payload length
- payload contents
- CRC init value

- gap cycles between bytes/bits
- back-to-back transaction spacing
- occasional reset insertion between transactions

Constraints:

- keep lengths within DUT-supported range
 - optionally bias toward boundary lengths:
 - 0
 - 1
 - 2
 - max-1
 - max
-

10. Scoreboard Checks

For each transaction:

- collect input payload and init value
- compute expected CRC using reference model
- compare with DUT `crc_out`
- report mismatch with:
 - payload
 - init value
 - expected CRC
 - actual CRC

Also check protocol behavior:

- `done` timing
 - output validity
 - no unknown (`x/z`) values on valid output
-

11. Functional Coverage

Suggested coverpoints:

Payload Length

- zero length
- minimum
- small

- medium
- maximum

Payload Pattern

- all zeros
- all ones
- alternating
- random
- incrementing

CRC Init Value

- default BLE init
- zero
- all ones within 24 bits
- random values

Reset Scenarios

- reset at idle
- reset between transactions
- reset during active transaction

Cross Coverage

- payload length × pattern
 - payload length × init value
 - reset scenario × transaction state
-

12. Pass/Fail Criteria

The DUT passes when:

- all directed tests pass
 - randomized regression completes with no mismatches
 - functional coverage reaches target, such as:
 - 100% for key directed bins
 - 90%+ for overall functional coverage
 - no unexpected protocol or timing violations occur
-

13. Exit Criteria

Verification is complete when:

- all planned tests are implemented and passing
 - scoreboard shows zero unresolved mismatches
 - all critical coverage bins are hit
 - known corner cases are exercised
 - any bugs found are fixed and regression-tested
-

14. Future Extensions

Later you could extend this plan to include:

- assertions for interface protocol
 - error injection
 - performance/latency checks
 - formal checks for CRC state transitions
 - compliance testing against captured BLE packet examples
-

If you want, I can turn this into:

1. a **more formal table-style verification plan**, or
2. a **UVM test plan plus test matrix with test names, descriptions, and coverage links**.